

AWS EC2 + Flask 웹 서버 배포 실전 매뉴얼

Gunicorn / systemd / Nginx / 도메인 / HTTPS까지

대상 환경: Ubuntu 기반 AWS EC2, Flask 프로젝트 ~/PPAP-Web, 진입 파일 app.py

목표: EC2를 시작하면 Flask 서버가 자동 실행되고, 최종적으로 https://내도메인으로 접속할 수 있는 운영 환경 구성.

최종 구조

```
■■■ ■■■■  
|  
| HTTPS :443  
v  
Nginx  
|  
| 127.0.0.1:8000  
v  
Gunicorn  
|  
v  
Flask
```

0. 현재까지 완료된 상태

현재 구성에서는 EC2의 Public IPv4 주소로 HTTP 접속이 가능하며, Gunicorn과 Nginx가 구성되어 있다.

완료 항목: EC2 생성 및 접속, 프로젝트 준비, Python 가상환경, requirements 설치, Flask 실행 테스트, Gunicorn 실행 테스트, systemd 서비스 등록, Nginx Reverse Proxy 설정, HTTP 접속 확인.

중요: 172.31.x.x 형태의 주소는 EC2 내부 Private IP이다. 외부 PC에서 접속할 때는 EC2의 Public IPv4 또는 이후 연결할 Elastic IP를 사용한다.

1. EC2와 Flask 기본 준비

1 - 1. 프로젝트 폴더 이동

```
cd ~/PPAP-Web
```

1 - 2. 가상환경 활성화

```
source venv/bin/activate
```

1 - 3. 의존성 설치

```
pip install -r requirements.txt
```

프로젝트에 .env.example이 없다면 cp .env.example .env 명령은 사용하지 않는다. 필요한 환경변수가 있는 프로젝트라면 별도로 .env 파일을 생성해야 한다.

1 - 4. Flask 개발 서버 테스트

```
python app.py
```

외부 테스트를 위해 app.py에서 개발 서버를 직접 사용할 경우 다음과 같은 형태로 실행할 수 있다.

```
if __name__ == "__main__":  
    app.run(host="0.0.0.0", port=5000, debug=True, use_reloader=False)
```

단, 실제 운영에서는 Flask 개발 서버를 외부에 직접 노출하지 않고 Gunicorn을 사용한다.

2. Gunicorn 설치와 테스트

2 - 1. 설치

```
cd ~/PPAP-Web  
source venv/bin/activate  
pip install gunicorn
```

2 - 2. 실행 테스트

```
gunicorn --bind 127.0.0.1:8000 app:app
```

정상이라면 Listening at: http://127.0.0.1:8000 및 Booting worker 메시지가 표시된다.

app:app 의미: 앞의 app은 app.py 파일명, 뒤의 app은 app.py 안에 선언된 Flask 객체 이름이다.

테스트가 끝나면 Ctrl+C로 종료한다.

3. systemd에 Gunicorn 등록

이 단계가 완료되면 SSH 터미널을 닫아도 서버가 계속 실행되며, EC2가 재부팅되어도 Gunicorn이 자동으로 시작된다.

3 - 1. 서비스 파일 생성

```
sudo nano /etc/systemd/system/ppap.service
```

아래 내용을 입력한다.

```
[Unit]  
Description=PPAP Flask Gunicorn Server  
After=network.target  
  
[Service]  
User=ubuntu  
Group=www-data  
WorkingDirectory=/home/ubuntu/PPAP-Web  
ExecStart=/home/ubuntu/PPAP-Web/venv/bin/gunicorn --workers 2 --bind 127.0.0.1:8000 app:app  
Restart=always  
RestartSec=3  
  
[Install]  
WantedBy=multi-user.target
```

3 - 2. 적용 및 자동 시작

```
sudo systemctl daemon-reload  
sudo systemctl start ppap  
sudo systemctl enable ppap
```

```
sudo systemctl status ppap
```

상태가 Active: active (running)이면 성공이다.

3 - 3. 이후 자주 사용하는 명령

```
sudo systemctl restart ppap
sudo systemctl stop ppap
sudo systemctl start ppap
sudo systemctl status ppap
```

앱 코드나 설정을 수정한 뒤에는 보통 `sudo systemctl restart ppap` 으로 Gunicorn을 재시작한다.

4. Nginx 설치 및 Reverse Proxy

4 - 1. 설치

```
sudo apt update
sudo apt install nginx -y
sudo systemctl enable nginx
sudo systemctl status nginx
```

4 - 2. Nginx 사이트 설정

```
sudo nano /etc/nginx/sites-available/ppap
```

도메인이 없는 현재 단계에서는 다음과 같이 설정한다.

```
server {
    listen 80;
    server_name _;

    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

4 - 3. 사이트 활성화

```
sudo ln -s /etc/nginx/sites-available/ppap /etc/nginx/sites-enabled/ppap
sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t
sudo systemctl restart nginx
```

`nginx -t`에서 `syntax is ok / test is successful`이 나오면 정상이다.

4 - 4. 현재 접속 방법

```
http://EC2-Public-IPv4
```

Nginx가 80번 포트에서 요청을 받고 내부의 Gunicorn 8000번 포트로 전달하므로 주소에 `:5000` 또는 `:8000`을 붙일 필요가 없다.

5. AWS Security Group

최종 운영 시 인바운드 규칙은 보통 다음과 같이 구성한다.

종류	포트	소스	용도
SSH	22	My IP	서버 관리
HTTP	80	0.0.0.0/0	HTTP 및 인증서 발급/리다이렉트
HTTPS	443	0.0.0.0/0	최종 HTTPS 서비스

Gunicorn을 Nginx 뒤에서 사용할 경우 5000과 8000 포트는 인터넷에 공개할 필요가 없다.

6. EC2 재부팅/중지 후 확인

현재 systemd와 Nginx가 enable 상태라면 EC2가 부팅될 때 자동으로 실행된다.

```
sudo systemctl status ppap
sudo systemctl status nginx
```

둘 다 active (running)이면 웹 서버 구성은 정상이다.

단, 일반 Public IPv4는 EC2를 중지 후 다시 시작할 때 변경될 수 있다. 도메인을 안정적으로 연결하려면 다음 단계에서 Elastic IP를 사용하는 것이 좋다.

앞으로 진행할 도메인 + HTTPS 설정

7. Elastic IP 할당

도메인을 구입하기 전에 EC2가 사용할 고정 공인 IP를 준비한다.

AWS 콘솔 경로: EC2 → Network & Security → Elastic IP addresses → Allocate Elastic IP address

할당 후 해당 Elastic IP를 선택하고 Actions → Associate Elastic IP address → 현재 PPAP EC2 인스턴스를 선택하여 연결한다.

연결 후 브라우저에서 다음 주소가 기존 웹사이트를 표시하는지 확인한다.

```
http://Elastic-IP
```

Elastic IP는 AWS의 요금 정책에 따라 비용이 발생할 수 있으므로 사용하지 않는 주소는 방치하지 않는다.

8. 도메인 구입

Route 53, 가비아 등 원하는 등록기관에서 도메인을 구입한다. 예를 들어 example.com을 구입했다고 가정한다.

AWS 안에서 관리하고 싶다면 Route 53의 Registered domains에서 도메인을 검색하고 등록할 수 있다.

도메인 등록이 완료된 뒤 DNS 관리 화면에서 EC2의 Elastic IP를 가리키도록 설정한다.

9. DNS A 레코드 설정

루트 도메인과 www를 모두 사용한다면 다음 두 레코드를 만든다.

Type	Name	Value
A	@	EC2 Elastic IP
A	www	EC2 Elastic IP

DNS 반영에는 시간이 걸릴 수 있다. 반영 후 http://example.com 으로 현재 사이트가 표시되는지 먼저 확인한다.

10. Nginx에 실제 도메인 등록

```
sudo nano /etc/nginx/sites-available/ppap
```

기존 server_name ;; 부분을 실제 도메인으로 변경한다.

```
server {  
    listen 80;  
    server_name example.com www.example.com;  
  
    location / {
```

```
proxy_pass http://127.0.0.1:8000;
proxy_set_header Host $host;
proxy_set_header X-Real-IP $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;
}
}
```

```
sudo nginx -t
sudo systemctl reload nginx
```

example.com은 반드시 실제로 구입한 도메인으로 변경한다.

11. Certbot으로 HTTPS 적용

DNS가 EC2 Elastic IP를 정상적으로 가리키고 있고, Security Group의 80/443 포트가 열려 있는 상태에서 진행한다.

11 - 1. Certbot 설치

```
sudo apt update
sudo apt install certbot python3-certbot-nginx -y
```

11 - 2. 인증서 발급

루트 도메인과 www를 모두 DNS에 연결했다면:

```
sudo certbot --nginx -d example.com -d www.example.com
```

www를 사용하지 않는다면:

```
sudo certbot --nginx -d example.com
```

Certbot의 안내에 따라 이메일 등을 입력하고 HTTP → HTTPS 리다이렉트를 활성화한다.

11 - 3. 접속 확인

```
https://example.com
```

브라우저에서 인증서 경고 없이 사이트가 표시되면 HTTPS 구성이 완료된 것이다.

11 - 4. 인증서 자동 갱신 테스트

```
sudo certbot renew --dry-run
```

테스트가 성공하면 인증서 자동 갱신 준비까지 완료된다.

12. 최종 완료 후 운영 방법

최종적으로는 다음 흐름으로 동작한다.

```
EC2 Start
|
+--> systemd -> ppap.service -> Gunicorn
|
+--> Nginx
|
■■■■ -> HTTPS :443 -> Nginx -> 127.0.0.1:8000 -> Flask
```

따라서 매번 SSH에 접속해서 source venv/bin/activate 또는 python app.py를 실행할 필요가 없다.

코드 수정 후

```
cd ~/PPAP-Web
# git pull ■■■ ■■■ ■■■
sudo systemctl restart ppap
```

Nginx 설정 수정 후

```
sudo nginx -t
sudo systemctl reload nginx
```

13. 문제 발생 시 확인 순서

- 1) EC2가 Running인지 확인
- 2) Security Group의 80/443 규칙 확인
- 3) Unicorn 상태 확인

```
sudo systemctl status ppap
```

- 4) Unicorn 로그 확인

```
sudo journalctl -u ppap -n 100 --no-pager
```

- 5) Nginx 상태/문법 확인

```
sudo systemctl status nginx
sudo nginx -t
```

- 6) Nginx 오류 로그 확인

```
sudo tail -n 100 /var/log/nginx/error.log
```

- 7) 내부 Unicorn 응답 확인

```
curl http://127.0.0.1:8000
```

내부 curl은 되는데 외부 접속이 안 되면 Nginx, Security Group, DNS 쪽을 우선 확인한다.

14. 최종 체크리스트

- ☐ EC2 인스턴스 Running
- ☐ Elastic IP 연결
- ☐ 도메인 구매 완료
- ☐ DNS A 레코드가 Elastic IP를 가리킴
- ☐ Security Group 22/80/443 설정
- ☐ ppap.service active (running)
- ☐ ppap.service enabled
- ☐ nginx active (running)
- ☐ http://도메인 접속 확인
- ☐ Certbot 인증서 발급 완료
- ☐ https://도메인 접속 확인
- ☐ certbot renew - -dry-run 성공

☐ 외부 5000/8000 포트는 다음